



REINFORCEMENT LEARNING

AULA 5



Prof. Gian Carlo Brustolin



CONVERSA INICIAL

Quando enfrentamos problemas complexos de IA, a exemplo do aprendizado em ambientes estocásticos episódicos e sequenciais, os métodos tradicionais são altamente eficientes para equacionar a política ideal mesmo quando não temos certeza sobre os resultados das ações do agente. Esses métodos exigem, entretanto, certa personalização para cada problema enfrentado e apresentam certas limitações, dificultando generalizações amplas, o que justifica, até esse momento, a inexistência de bibliotecas de soluções para RL, por exemplo.

IA conexionista, em contrapartida, é uma aproximação, por definição, genérica, posto que firma suas bases sobre uma modelagem matemática, não do problema em si, mas da unidade mínima de pensamento, o neurônio. Arquitetada uma rede de neurônios, basta treinar tal rede para que nos forneça o resultado que esperamos.

Uma rede neural artificial não é, todavia, um ente computacional simples. Para que seja possível explorar seu potencial é necessário que entendamos como as diversas arquiteturas, configurações e métodos de treinamento agem.

Nesta etapa, vamos então passar pelos conceitos básicos de neurônio artificial, evoluindo para o entendimento dos processos de aprendizagem, memória e adaptação de redes de neurônios artificiais. Ao final, seremos capazes de entender as RNAs tradicionais e convolucionais envolvidas em aprendizagem profunda.

TEMA 1 – NEURÔNIO ARTIFICIAL

A definição definitiva de pensamento e inteligência ainda são incertas. Estudamos IA sem de fato podermos conceituar o que é inteligência, seja esta artificial ou não. A melhor aproximação foi feita por Turing ao final da Segunda Guerra Mundial, mas ela apenas avalia se uma máquina apresenta ou não comportamento inteligente sem tangenciar a definição de inteligência propriamente dita.

Os primeiros passos na tentativa de produzir artificialmente a inteligência se deram no sentido da tentativa de reprodução do cérebro humano. O que se pretendia é sintetizar o pensamento pela reprodução da fisiologia cerebral. Essa



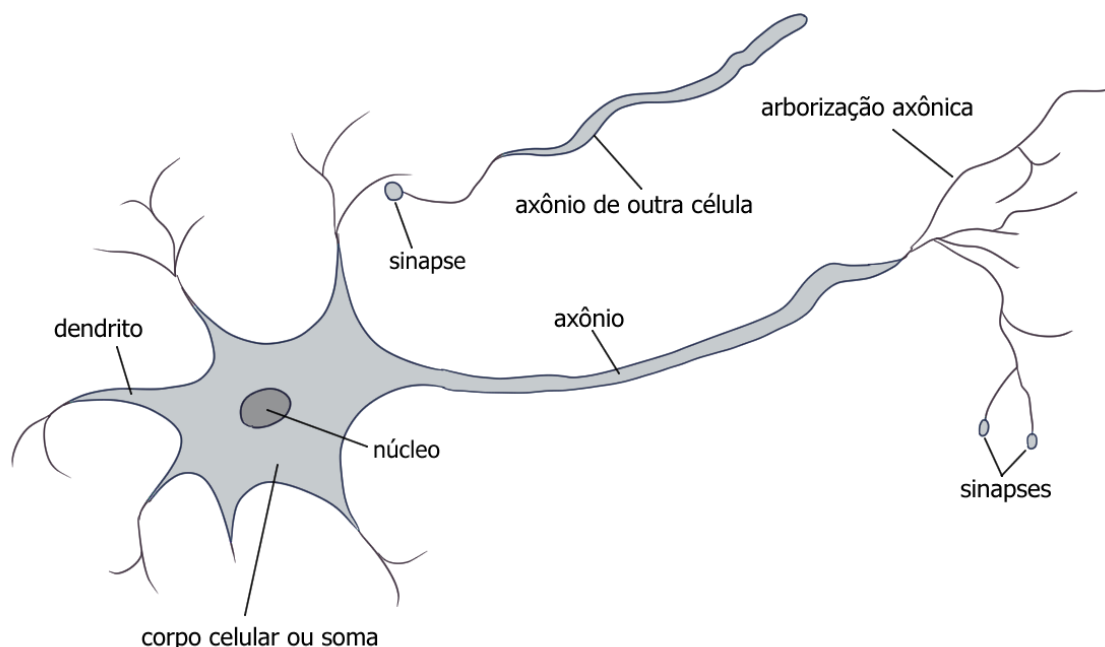
técnica foi abandonada após poucos anos de pesquisa em razão da incapacidade de processamento dos equipamentos da época. Foi retomada com sucesso, entretanto, no final do século passado.

1.1 Neurônio biológico

As pesquisas médicas sobre a fisiologia do cérebro e a comprovação, em 1911, por Ramon y Cajál, da existência das células nervosas cerebrais, orientaram as conclusões dos matemáticos da época no sentido de sintetizar as funções de um neurônio em uma modelagem numérica.

Para que entendamos tal modelagem, devemos evoluir um pouco mais em nosso conhecimento da biologia cerebral. Na Figura 1 a seguir, ilustramos um neurônio genérico composto por um único corpo celular. As observações dos primeiros neurologistas determinaram que um neurônio recebe sinais eletroquímicos de outros neurônios por meio de seus dendritos e transmite o “processamento” dos sinais recebidos por meio de sua arborização axônica. O córtex cerebral é constituído por várias regiões e cada região é composta de neurônios com características biológicas ligeiramente distintas.

Figura 1 – Neurônio genérico



Crédito: Jackeline Souza.

Genericamente, entretanto, um neurônio pode receber informações de muitos outros neurônios da mesma forma que transmite a síntese dessas



informações para outra infinidade de pares. Segundo Haykin (2011, p. 33), o número de interações ou conexões para um único neurônio pode chegar a 10.000 e as conexões de uma única célula neural montam também aos milhares. A região de proximidade entre um dendrito de uma célula e a ramificação do axônio de outra recebeu o nome de sinapse.

A transmissão do estímulo entre neurônios se dá por pulsos de tensão, originados no núcleo da célula, ditos *spikes*. Os *spikes* podem viajar entre conjuntos de neurônios, sofrendo ampliação ou decaimento conforme a proximidade entre neurônios e o tratamento que o núcleo da célula neural dá aos impulsos entrantes, antes de retransmiti-los para as sinapses de sua árvore axônica.

As sinapses representam o nível mais fundamental da atividade cerebral, e as árvores dendritais são o próximo nível de “processamento”. Nesse nível, os estímulos advindos das células vizinhas são associados.

Um tipo de neurônio dito piramidal, por exemplo, apresenta três árvores dendritais, duas basais e uma apical. Nesse tipo de célula neural, os estímulos sinápticos são combinados em cada árvore antes de serem enviados ao núcleo.

O próximo nível de processamento da informação ocorre em agrupamentos neurais. Esses agrupamentos de células parecem realizar tarefas específicas, e o agrupamento é sempre estimulado quando dada tarefa é requisitada ao cérebro.

Tarefas complexas podem requisitar o envolvimento de tipos diversos de neurônios. Nesse caso, circuitos inter-regionais são criados, conectando neurônios de morfologias distintas pertencentes a regiões diversas do córtex.

Essa enorme flexibilidade neural é típica dos tecidos cerebrais e não encontra paralelo no restante da citoarquitetura humana.

As redes neurais artificiais estão muito distantes da capacidade de reproduzir essa arquitetura complexa, mas mesmo com uma modelagem simplória conseguem importar importantes características do comportamento inteligente.

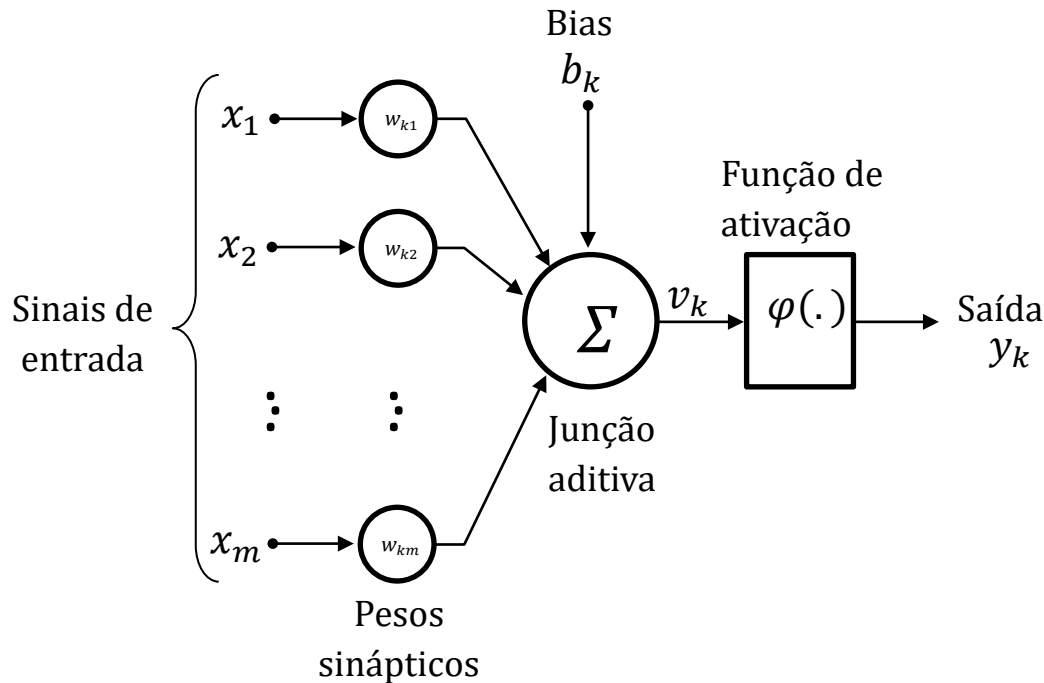
1.2 Modelagem matemática do neurônio

Agora que conhecemos, de maneira bastante básica, a morfologia neural natural, podemos apresentar o modelo matemático que, mesmo de forma



rudimentar, representa esta citoarquitetura. Um neurônio artificial normalmente é modelado segundo a Figura 2 a seguir.

Figura 2 – Modelo de neurônio



Fonte: Haykin, 2011, p. 36.

A árvore dendrítica é representada pelo vetor de entrada $X = \{x_1, x_2, \dots, x_m\}$ e o axônio pela saída “y”. Cada elemento do vetor X tem um peso neural “w” associado, que simulará a proximidade entre neurônios, ou seja, a intensidade da sinapse; por esse motivo recebem o nome de “pesos sinápticos”. Os pesos sinápticos também podem ser representados na forma de um vetor de forma análoga às entradas $W = \{w_1, w_2, \dots, w_m\}$.

O combinador linear, ou somador, modela o processamento da árvore dendrítica. A função de ativação simula o tratamento que o núcleo da célula nervosa dispensa ao estímulo elétrico recebido dos dendritos.

O bias “b” foi inicialmente imaginado como modelagem possível do processamento de diversas árvores dendríticas. Percebeu-se, entretanto, que o bias auxilia no ajuste prévio do nível do somador, antes de entregarmos o resultado para a função de ativação.

O resultado do somatório dos produtos de X e W é dito campo local induzido ou potencial de ativação, v_k .

A saída do neurônio deverá excursionar entre 0 e 1 ou entre -1 e $+1$.

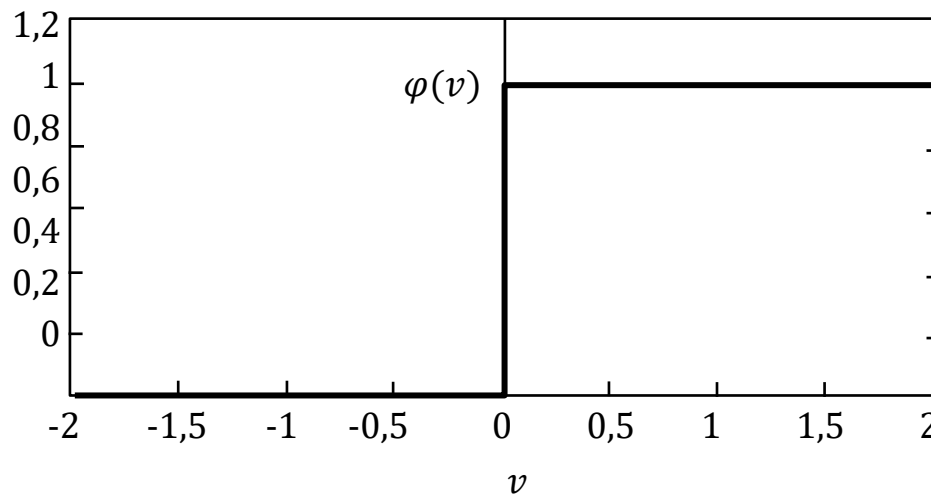


Quanto à função de ativação, essa equação matemática modelará a reação do núcleo celular a resultante dos estímulos, como já citamos. Para dar generalidade ao modelo, algumas funções de ativação foram propostas pela literatura, conforme descreveremos a seguir.

O primeiro modelo neural, proposto por McCulloch e Pits, em 1943, utilizava uma função de ativação limiar (degrau unitário ou Heaviside) descrita pela equação a seguir, cuja representação gráfica está disposta na Figura 3 a seguir.

$$\varphi(v) = \begin{cases} 1 & \text{se } v \geq 0 \\ 0 & \text{se } v < 0 \end{cases}$$

Figura 3 – Função de ativação limiar



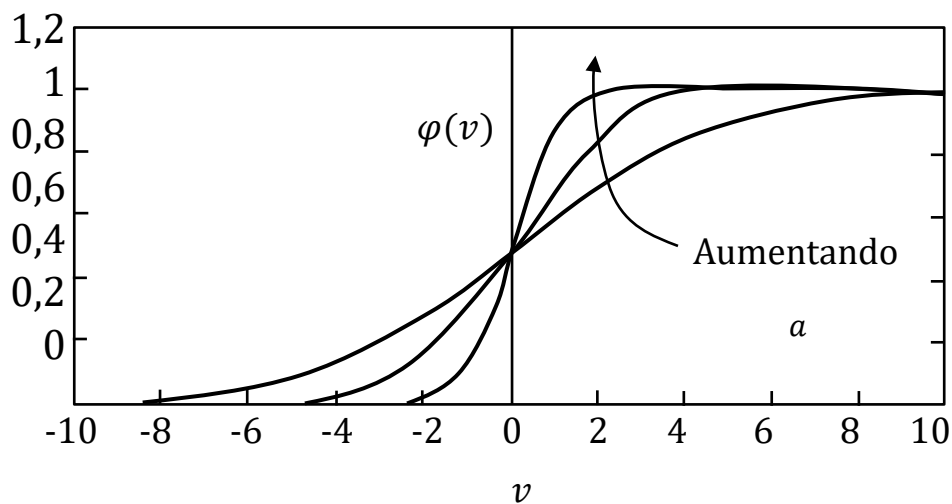
Fonte: Haykin, 2011, p. 39.

Essa função de ativação foi a primeira a ser proposta. O modelo de McCulloch e Pits recebeu a designação de Perceptron e tem utilização nas redes neurais atuais. A função limiar tem a desvantagem de restringir “y” a valores discretos.

A função sigmoide, que pode ser vista como uma suavização da função limiar, normalmente conduz a melhores resultados por possibilitar a “y” excursionar de forma não discreta entre os níveis 0 e 1, como pode ser visto na Figura 4 a seguir.



Figura 4 – Função de ativação sigmoide



Fonte: Haykin, 2011, p. 39.

A equação dessa curva é dada pela seguinte equação:

$$\varphi(v) = \frac{1}{1 + \exp(-av)}$$

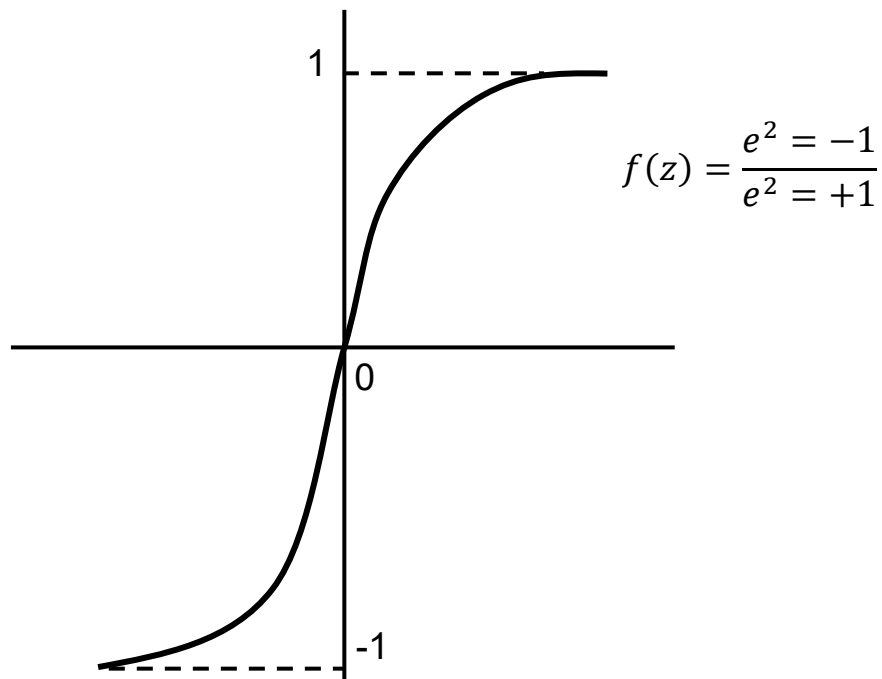
Essa equação pode sofrer uma generalização ainda maior de parâmetros se a representarmos pela equação genérica a seguir, por vezes chamada de função *softmax*:

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

Observando a equação anterior, podemos intuir a aproximação com um cálculo de probabilidade. De fato, a função *softmax* pode ser aplicada para transformar a saída em uma distribuição probabilística.

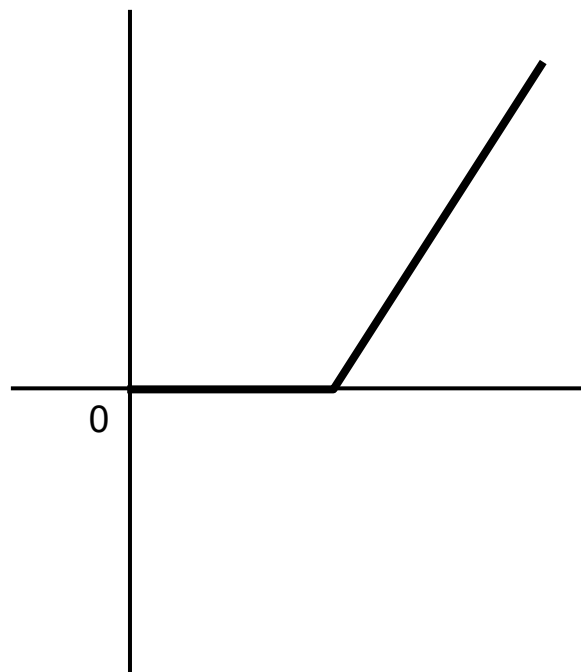
Antes de seguirmos com mais algumas funções de ativação, é importante comentar que esse modelo, por nós estudado até o momento, contempla um modelo determinístico de neurônio. Pode ser conveniente, em alguns casos, encarar o disparo do neurônio como um evento incerto e, nesse caso, a função de ativação pode acrescentar essa incerteza em sua equação.

Retomando às funções de ativação, devemos ainda conhecer a função tangente hiperbólica que tem aplicação mais restrita, mas merece ser citada:



Crédito: Brustolin 2022/UT

Finalmente, em exercícios empíricos mais recentes, a retificação de uma função linear, dita ReLU ou *Rectified Linear Unit*, parece se enquadrar bem em processos de aprendizagem profunda, nas camadas ocultas (tema ao qual voltaremos em breve). A seguir, podemos ver a representação gráfica:

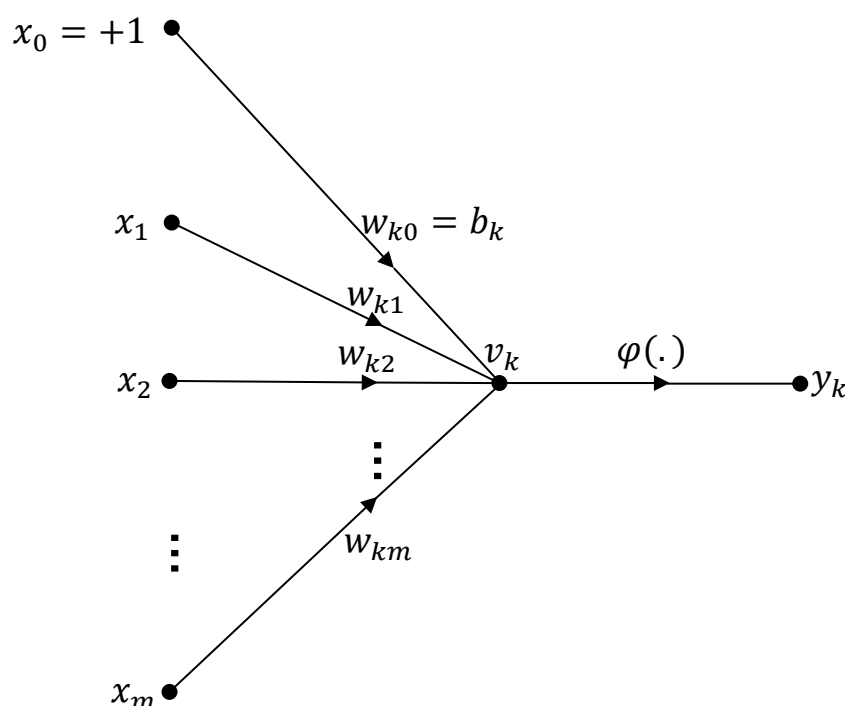


Crédito: Brustolin 2022/UT

1.3 Grafo do neurônio artificial

Especialmente importante, quando se intentar o uso de aplicações ou bibliotecas de redes neurais, a exemplo do TensorFlow, a representação dos neurônios e suas interconexões como grafos orientados pode ser utilizada para simplificar a apresentação esquemática da rede. Na Figura 5 a seguir, podemos visualizar o grafo clássico de representação de um neurônio.

Figura 5 – Grafo de um neurônio



Fonte: Haykin, 2011, p. 43.

É interessante observar que, para maior generalização da representação, preferiu-se tornar o bias um dos componentes do vetor X . Isso se faz tomando um elemento $x_0 \in X$ de forma que seu valor seja sempre considerado positivo e unitário. Feita essa consideração, podemos associar o bias ao peso sináptico dessa entrada como $w_0 \in W$.

TEMA 2 – APRENDIZAGEM, MEMÓRIA E ADAPTAÇÃO

No tópico anterior, entendemos como é possível modelar um neurônio e simular a intensidade de suas conexões sinápticas com os neurônios vizinhos por meio dos pesos sinápticos. Já conhecemos as bases de algumas técnicas de treinamento, de um neurônio artificial ou de uma coleção de neurônios, para



que realizem determinada atividade. A seleção da técnica de aprendizagem dependerá do tipo de atividade, ou tarefa, a ser realizado pelo arranjo neural. Além disso, após o treinamento, devemos entender como a memória neural pode ser mantida e como podemos tornar esse aprendizado mais flexível.

2.1 Tarefas de aprendizagem

Segundo Haykin (2011, p. 91) seis são as tarefas para as quais podemos treinar uma rede neural, quais sejam: associação de padrões, reconhecimento de padrões, aproximação de funções, controle, filtragem e formação de feixe.

Na associação de padrões, a rede deve aprender recuperar um padrão ruidoso, ou seja, memorizará vetores padronizados e, ao receber um vetor ruidoso, distorcido em relação aos padrões de treinamento, saberá recuperá-lo, associando-o a um dos vetores padrão. Dessa forma, a saída será também um vetor e não apenas um número.

No reconhecimento de padrões, a saída será uma classe, ou seja, a rede neural classificará o vetor de entrada como pertencente a um número finito de classes. Essa tarefa tem contornos estocásticos, posto que é capaz de criar distribuições ou espaços de classificação de eventos. Máquinas de aprendizagem profunda são baseadas nesse princípio.

A aproximação de funções é uma tarefa de aprendizagem na qual a máquina se torna capaz de reproduzir um mapeamento entrada-saída em função de amostras. Nessa tarefa, não se conhece a relação matemática entre X e y mas sabe-se existir $f(X)=y$. A rede treinada de forma supervisionada com pares (X,y) se torna capaz de, ao receber um vetor desconhecido X_i , reproduzir a função $y_i=f(X_i)$.

A tarefa de controle refere-se à análise de indicadores de uma planta ou processo qualquer com a finalidade de ajustar os parâmetros desse processo para que mantenha os indicadores dentro de determinados níveis.

Na filtragem, o treinamento com padrões não ruidosos confere à rede a capacidade de, ao receber o padrão acrescido de ruídos, eliminá-los, retornando ao padrão não ruidoso original. A habilidade para essa tarefa torna as redes neurais bastante importante para o tratamento digital de sinais.

Um tipo especial de filtragem é a chamada “formação de feixe”, que trata de identificar um feixe sonoro com o objetivo de localizar ou identificar dado



objeto. Esse tipo de filtragem é usado em radares e pode ser objeto de aprendizagem de uma rede neural.

2.2 Redes neurais e memória

Entendemos quais as tarefas realizáveis por um agrupamento ou mesmo por um único neurônio. Vamos agora conceituar esses agrupamentos e entender como o treinamento pode ser memorizado.

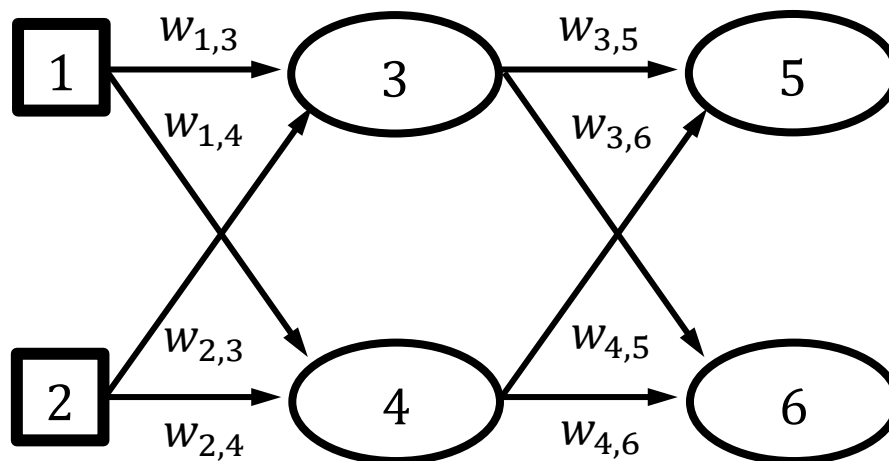
A evolução da neurociência computacional criou novos modelos de neurônios que mais se aproximam da funcionalidade das células cerebrais, entretanto, o modelo simples de neurônio, como descrito em nosso tópico anterior, demonstrou ter características interessantes quando encadeado com outras entidades semelhantes. As redes neurais artificiais baseiam suas unidades de formação nesse modelo elementar estudado (Norvig, 2013, p. 635).

Quando conceituamos pesos sinápticos, dissemos que esses valores modelam a força da conexão neural. Assim, ao associarmos neurônios artificiais, a saída de um neurônio " y_{k-1} " será conectada à entrada $x_{i,k}$ do próximo neurônio " k ". Quando a entrada " $x_{i,k}$ " for processada pela função de ativação " ϕ_k ", receberá uma apreciação ou depreciação em relação às demais entradas, em função do peso sináptico " $w_{i,k}$ ". Esse processo possibilita a associação de muitos neurônios de forma aleatória do ponto de vista de arquitetura de conexão.

Fundamentalmente duas são as formas de associar neurônios artificiais: **redes alimentadas adiante**, na qual não há laços de recorrência, e **redes recorrentes**. Redes recorrentes podem apresentar problemas de estabilidade, uma vez que suas saídas são reinseridas na rede, possibilitando a oscilação, se um bom controle não for implementado.

As redes alimentadas adiante, por sua vez, têm arquiteturas dispostas em camadas. A camada que recebe o vetor de entrada é dita camada de entrada. A camada interior da rede, se houver, é chamada de camada oculta; já a camada formada pelos neurônios de saída será denominada camada exposta ou de saída. **Redes alimentadas adiante se totalmente conectadas são também denominadas MLPs (MultiLayer Perceptron)**. No esquema da Figura 6 a seguir, ilustramos uma MLP com camada de entrada (neurônios 1 e 2), camada oculta (neurônios 3 e 4) e camada de saída (neurônios 5 e 6).

Figura 6 – Rede neural alimentada adiante



Fonte: Norvig, 2013, p. 637.

Quando todas as entradas são conectadas diretamente à saída (ou seja, quando não há camada oculta), a rede é chamada de RNA de camada única ou rede *perceptron* simples. Como observamos, nas denominações de redes, a camada de entrada não é computada. Na Figura 4 anterior, por exemplo, temos uma rede de duas camadas e não três. Isso ocorre porque, normalmente, se designa um neurônio na camada de entrada para cada x_i sem que esse neurônio interfira no sinal de entrada.

O treinamento de uma rede neural alimentada adiante é feito pela alteração de seus parâmetros livres, ou seja, pelo ajuste dos pesos sinápticos e eventualmente do bias. A função de ativação de cada neurônio é definida antes do início do treinamento e não sofrerá alterações durante a aprendizagem.

Uma vez treinada, a rede será capaz de executar determinada tarefa, portanto, é capaz de armazenar ou memorizar o algoritmo associado à tarefa. A esse fato se nomina memória associativa distribuída, uma vez que é composta efetivamente pela coleção dos parâmetros livres, distribuídos, ajustados da rede.

Haykin (2011, p. 101) afirma que a memória associativa de uma RNA é capaz de realizar um mapeamento distribuído entre a atividade do espaço vetorial de entrada para um espaço de saída determinado e controlável.

Seguindo as conclusões daquele autor, para conceituarmos matematicamente a memória de uma RNA, vamos sintetizar uma rede por apenas um neurônio que realiza a transdução entre entrada e saída, que segue



o modelo da Figura 5 anterior. Partindo da suposição de que temos uma função de ativação linear, é possível considerar a saída da rede de dimensão “m” como:

$$y_{ki} = \sum_{j=1}^m w_{ij}(k)x_{kj}, \quad i = 1, 2, \dots, m$$

Para que essa equação seja válida, aceitamos a dimensionalidade da rede como dada pela dimensão “m” do vetor “X” de entrada. Igualmente aceitamos que a dimensão do vetor Y, de saída, será a mesma de “X”. Outra forma de vermos a mesma equação será na representação matricial:

$$\begin{bmatrix} y_{k1} \\ y_{k2} \\ \vdots \\ y_{km} \end{bmatrix} = \begin{bmatrix} w_{11}(k) & w_{12}(k) & \dots & w_{1m}(k) \\ w_{21}(k) & w_{22}(k) & \dots & w_{2m}(k) \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}(k) & w_{m2}(k) & \dots & w_{mm}(k) \end{bmatrix} \begin{bmatrix} x_{k1} \\ x_{k2} \\ \vdots \\ x_{km} \end{bmatrix}$$

A matriz “W” constitui a memória da rede, que é recursivamente treinada para absorver o conhecimento. Quando o erro de mapeamento se torna pequeno, interrompe-se o treinamento e nominaremos a matriz de pesos ajustados como M, designando a memória associativa adquirida pelo treinamento. Podemos então escrever:

$$\mathbf{M} = \mathbf{Y} \mathbf{X}^T$$

Conforme acabamos de entender, a memória da rede M é capaz de mapear a entrada X para a saída Y. Porém, a obtenção de M é feita de forma iterativa pelo ajuste de W a cada vetor de entrada. Dessa forma, a memória será capaz de reconhecer os vetores de treinamento, mas com um “ruído”. Esse ruído é proveniente da presença, em M, da memória cumulativa dos demais padrões vetoriais de entrada.

Imaginemos agora que a rede recebe um vetor $X_{\text{nov}}o$ desconhecido da rede, mas próximo de um padrão de treinamento X_i qualquer. É provável que a rede interprete $X_{\text{nov}}o$ como se X_i fosse. A isso denominamos **capacidade de generalização da rede**.

Se medirmos o ângulo espacial entre $X_{\text{nov}}o$ e X_i podemos determinar qual o ponto de alteração da expectativa de saída da rede, mas esse ângulo não será o mesmo para todos os $X_{\text{nov}}o$, uma vez que a escolha dos padrões (amostras) de treinamento é aleatória. De qualquer forma, a generalização (e, portanto, o aprendizado da rede) é um fenômeno estocástico. Existe uma probabilidade,



eventualmente calculável, da rede generalizar corretamente, mas não há certeza do mapeamento correto para um vetor aleatório X entre entrada e saída. Vamos examinar esse tema com um pouco mais de detalhes a seguir.

2.3 Teoria estatística da aprendizagem

Já entendemos que uma rede neural é capaz de memorizar um mapeamento entre entrada e saída, mesmo que esse mapeamento não seja plenamente conhecido por quem a treina. Dito de outra forma, suponha que a relação entre o espaço de entrada X e o espaço de saída D possa ser escrito como $D = F(X)$, desconhecida. A rede neural é capaz de memorizar uma relação $D = f(X, W)$ que se aproxima de $F(X)$, mas não será idêntica a $F(X)$ por hipótese.

Essa “maldição” ocorre, como já estudamos, em função do ruído inserido na predição de X_i pelos outros vetores de treinamento pertencentes mesmo ao espaço vetorial $\mathcal{X}$ a que X_i pertence. Dessa forma, poderíamos escrever:

$$D = f(X, W) + \varepsilon$$

Em que ε designa o erro esperado na estimativa de “ D ” em função de “ X ” durante o treinamento.

De certa forma, sem nos delongarmos em cálculos matemáticos, podemos intuir que a parte linear de ε pode ser ajustada pelo parâmetro do bias.

Terminado o treinamento, espera-se que ε tenda a zero, embora possa-se pensar que há um limite no número de amostras $\hat{A}$ que uma rede pode aprender para que ε permaneça “suficientemente nulo”, ou seja, tão baixo que possa ser considerado nulo. De fato, esse parâmetro é chamado de dimensão V-C. O cálculo de V-C é normalmente intratável, mas pode-se imaginar que há certa proporcionalidade entre V-C e a dimensão de W , mesmo que não linear.

Supondo que se obtenha $\hat{A}$ eliminando o erro de aprendizagem ε , não há garantia de que as amostras escolhidas, embora na quantidade correta, minimizarão o erro quando a rede entrar em modo de generalização.

Uma “boa amostragem”, entretanto, pode reduzir ε , uma vez que potencialmente reduzirá o ângulo máximo entre os vetores padrão e o vetor qualquer de entrada melhorando a estimativa de Y (saída do neurônio). Assim:

$$Y = f(X, W) + \varepsilon g$$



É de se esperar que Y se aproxime de D , embora, da mesma forma que dissemos anteriormente, provavelmente não ocorrerá $Y=D$ exceto para $\hat{A}$, permanecendo um erro $\mathcal{E}_g$, como resultado da presença do ângulo espacial entre a amostra de treinamento e o vetor de entrada real.

A esse erro que se aproxima de Y por $f(X,W) + \mathcal{E}_g$ denominaremos erro de generalização. Podemos imaginar uma região de crença na qual encontraremos D ; esta será delimitada por $\mathcal{E}$ em torno de Y . Essas conclusões tornam o processo de aprendizagem fortemente estocástico.

Compreendido esse paradigma, podemos mergulhar em uma técnica de treinamento para redes neurais que leva em conta a correção iterativa do erro de treinamento.

TEMA 3 – TREINAMENTO DE MLP E CONVERGÊNCIA

A aprendizagem de uma rede neural artificial alimentada adiante, MLP, como já podemos intuir por tudo que comentamos a respeito dessas entidades até o momento, é feita por métodos iterativos. Esses algoritmos de treinamento alteram os pesos sinápticos de forma dinâmica, controlando o erro, no caso de métodos supervisionados, ou a estabilidade desses parâmetros, no caso de aprendizagem não supervisionada. Neste tema, entenderemos como realizamos o treinamento supervisionado de uma RNA MLP.

3.1 Épocas de treinamento

Em nossa seção anterior, descobrimos que, para realizar o treinamento de uma MLP de forma supervisionada, devemos escolher, no espaço de operação da rede, pares de vetores X e D , de forma que a rede aprenda o mapeamento

$D = f(X,W)$. Dito de outra forma, devemos amostrar o espaço de eventos e ajustar W para que a saída da MLP corresponda a D . Também aprendemos que esse mapeamento, em função do ruído, apresentará um erro de treinamento $\mathcal{E}$.

Citamos a possibilidade de se reduzir $\mathcal{E}$ pela apresentação reiterada da coleção amostral à MLP, ajustando-se, a cada iteração, W . Esse processo iterativo de repetir o ajuste de W com a mesma amostra é chamado de **época de treinamento**.



Vamos supor que temos 50 amostras de um mapeamento hipoteticamente infinito. A rede pode ser treinada em uma primeira época pelo ajuste de W para essas 50 amostras. Verificado $\mathcal{E}$, pode ser necessário um novo ajuste (ou uma nova época de treinamento). Essa segunda época será realizada com as mesmas 50 amostras.

As épocas se repetirão até que se atinja $\mathcal{E}$ suficientemente pequeno.

3.1 Superfície de erro

Ao observarmos a evolução do erro $\mathcal{E}$ durante as épocas de treinamento, esperamos, intuitivamente, um comportamento linear, de forma que mostre um valor menor a cada iteração $\mathcal{E}$. Esse, entretanto não é o comportamento típico de $\mathcal{E}$.

Haykin (2011, p. 261) afirma que a superfície de $\mathcal{E}$ para um MLP é uma função altamente não linear. Assim, não devemos esperar linearidade alguma na evolução de $\mathcal{E}$. Podemos prever a existência de mínimos locais e platôs, ou seja, precisaremos de muita matemática ou boa dose de sorte para encontrarmos um mínimo global para $\mathcal{E}$. A Figura 7 a seguir exemplifica uma distribuição típica do erro em função das sequentes iterações.

Figura 7 – Superfície de erro em RNA



Crédito: Brustolin 2022/UT

Dessa forma, será necessário controlar a evolução de $\mathcal{E}$, mas já sabemos de antemão que, dada a não linearidade, a quantidade de iterações não tem



relação direta com a redução de $\mathcal{E}$. Dito de outra forma, podemos ter um resultado ótimo para 10 épocas e um resultado muito pior para 10.000 épocas.

3.2 Gradiente descendente

Gradiente descendente é uma metodologia de busca local de derivadas negativas em espaços não lineares. Por ser uma busca local, esses algoritmos não são necessariamente ótimos, ou seja, somente serão completos se o ponto de partida for factível (Castoldi, 2011, p. 74). Para que entendamos a metodologia, devemos visitar o conceito matemático de gradiente de forma bastante singela.

Suponhamos uma função multivariada $\varnothing = f(x, y, z, \dots)$. Se desejarmos derivar $\varnothing$, devemos escolher a variável em função da qual a derivada será calculada. Assim, podemos derivar $\varnothing$ parcialmente em função de x , de y , de $z, \dots$ e representamos a derivada parcial de $\varnothing$ em função de x como $\partial\varnothing/\partial x$. O gradiente de $\varnothing$, $\nabla\varnothing$, será um vetor no espaço vetorial de $\varnothing$, formado por todas as derivadas parciais de $\varnothing$. Podemos escrever:

$$\nabla\varnothing = [\partial\varnothing/\partial x, \partial\varnothing/\partial y, \partial\varnothing/\partial z, \dots]$$

Saiba mais

Mais detalhes sobre a teoria matemática envolvida podem ser vistos em McCallum (2017, cap. 3).

O interessante sobre $\nabla\varnothing$ é que ele é um vetor que aponta no sentido de crescimento da função $\varnothing$. Dessa forma, se buscamos o mínimo de uma função $\varnothing$, basta caminharmos em direção contrária a $\nabla\varnothing$.

Voltando ao problema do erro no ajuste dos pesos sinápticos. Supondo que o “ d_j ” represente o valor desejado na saída “ y_j ” de um neurônio “ j ” da camada exposta de uma rede neural, podemos escrever sobre o sinal de erro após a n ésima iteração:

$$\mathcal{E}(n) = d_j(n) - y_j(n)$$

Vamos conceituar a energia do erro $\mathcal{E}(n)$ em uma rede neural com a metade da variância de $\mathcal{E}(n)$. Isso é útil porque ocorrem erros positivos e



negativos durante o treinamento e, se simplesmente os somássemos para obter a estimativa do erro total, por intermédio do desvio da média, os erros negativos enviesariam a soma. Assim:

$$C(n) = \frac{1}{2} \sum_{j \in C} \varepsilon_j(n)^2$$

A energia média pode ser encarada como o **custo do treinamento**, uma vez que seria obtida pela média aritmética de todas as n energias de erro durante dado treinamento. Um treinamento mais eficiente terá uma C média menor que outro algoritmo de menor eficiência.

Justo seria imaginar que o gradiente de C nos indicará a direção de minimização do erro. Uma aplicação prática desse conceito se dá pela retropropagação do erro de forma a corrigir os pesos sinápticos da rede.

O **custo do treinamento** pode ser estimado por várias outras formas além da energia média do erro. Uma forma interessante para camadas de saída não discretas é medir a convergência entre o valor de y_a em uma época em relação à próxima predição, $\hat{y}_a$. Essa técnica contorna o problema do enviesamento da somatória dos erros, que já comentamos, uma vez que não avalia diretamente o erro, mas sim a estabilidade da saída, ou entropia da saída. De forma a considerar a aproximação não linear da derivada nula, o custo utiliza o logaritmo da predição $\hat{y}_a$. Escrevemos:

$$\text{Custo} = - \sum y_a * \log(\hat{y}_a)$$

Essa função é chamada de “custo de entropia cruzada”. Da mesma forma, podemos buscar a minimização dessa função por gradiente descendente.

3.3 Retropropagação do erro

Entendemos que o cálculo do gradiente do erro nos possibilita uma forma segura de controlá-lo. O erro, por sua vez, em uma rede neural, depende basicamente do ajuste que se faz dos pesos sinápticos. Podemos então pensar que o gradiente parcial em relação a “ w ” nos fornecerá a direção de minimização do erro em função dessa variável. Assim, será possível corrigir o peso w_n por um incremento Δw que pode ser calculado pelo gradiente, como segue:



$$\Delta w = - \partial C / \partial w$$

Pode ser interessante suavizar o ajuste de w_n , dada a não linearidade da superfície de erro, por um fator que chamaremos de “taxa de velocidade de aprendizagem”, ou “taxa de aprendizagem α ”. Como a superfície tem características fortemente não lineares, um ajuste abrupto de w_n pode conduzir à próxima iteração para fora da área de descida, tonando o erro maior. Dessa forma, é mais conveniente ajustar Δw pela equação:

$$\Delta w = \alpha \partial C / \partial w$$

Derivar o erro significa derivar a diferença entre $\mathbf{d}$ e $\mathbf{y}$; por sua vez, $\mathbf{y}$ pode ser escrito como uma função de $\mathbf{x}$, $\mathbf{w}$ e da função de ativação do neurônio. Assim, ao aplicarmos a derivada, eliminamos as constantes e terminaremos com a derivada da função de ativação. Haykin (2011, p. 193) demonstra que uma boa forma de aproximar $\partial C / \partial w$ é pela multiplicação do gradiente local de cada neurônio pelo sinal de entrada. Assim, podemos escrever:

$$\begin{pmatrix} \text{Correção} \\ \text{de peso} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{taxa de aprendizagem} \end{pmatrix} \cdot \begin{pmatrix} \text{Gradiente} \\ \text{local} \end{pmatrix} \cdot \begin{pmatrix} \text{sinal de entrada} \\ \text{do neurônio } j \\ y_i(n) \end{pmatrix}$$

No que se refere ao gradiente local, Haykin demonstra que seu cálculo pode ser feito pela multiplicação da derivada da função de ativação pelo sinal de erro do neurônio. Tomando o gradiente por δ e a derivada da função de ativação do neurônio como φ' , podemos escrever:

$$\delta = \varphi'(v) * \varepsilon$$

Em que “ v ” é o valor do campo local para a entrada X .

Essa conclusão é facilmente aplicada na camada exposta, em que o erro de cada neurônio de saída corresponde ao erro da rede neural.

Essa correção, entretanto, não é suficiente para ajustar a memória da rede ao padrão; será necessário propagar o ajuste de w_n para as camadas anteriores a de saída. Esse processo será dito **retropropagação do erro** (ou BP, *Back Propagation*, em inglês).



O ajuste de w_n por Δw nas demais camadas anteriores a de saída não pode basear-se no erro, já que não há como conhecer o erro do neurônio, apenas o erro da saída da rede. Nesses casos, o uso da soma ponderal dos gradientes locais da camada posterior, conectada ao neurônio, substituirá o erro, possibilitando o cálculo.

3.4 Critério de parada

As iterações para ajuste de W devem ser suspensas em dado ponto. Já ficou claro que o número de iterações não é um bom critério. Podemos ter um resultado ótimo para uma dezena de épocas e um resultado muito pior para milhares de épocas. Assim, a parada deve ser definida pela estabilidade da energia do erro, ou seja:

$$\partial \epsilon / \partial w = 0$$

Como a descrição da equação de ϵ não é facilmente obtida, o cálculo da derivada literal também não será possível. Normalmente, uma variação entre iterações para o valor de **$\epsilon < 1\%$ é um bom indício de que estamos nos aproximando da região de derivada nula**. Dessa forma, já somos capazes de treinar de forma supervisionada uma MLP e sabemos em que ponto devemos concluir o treinamento.

As MLPs, como afirmamos anteriormente, são entidades estáveis temporalmente, por não possibilitarem a realimentação de parâmetros. Se neurônios da rede recebem realimentação de camadas posteriores a ele, a rede terá uma memória de curta duração diversa do comportamento em regime. Vamos estudar este tópico em seguida.

TEMA 4 – PROCESSAMENTO TEMPORAL

Até esse momento, concentramos nossa atenção em redes nas quais o processamento depende exclusivamente da memória obtida pelo treinamento. Nas arquiteturas de redes alimentadas adiante, estudadas, treinamos a rede até que o gradiente de erro se torne muito próximo de zero, e então encerramos o treinamento e colocamos a rede para operar, ou seja, para que generalize o aprendizado. Essas redes podem ser ditas atemporais, uma vez que não reagem



ao tempo, ou seja, não reagem ao momento no qual o padrão ou o vetor de teste percorre a rede.

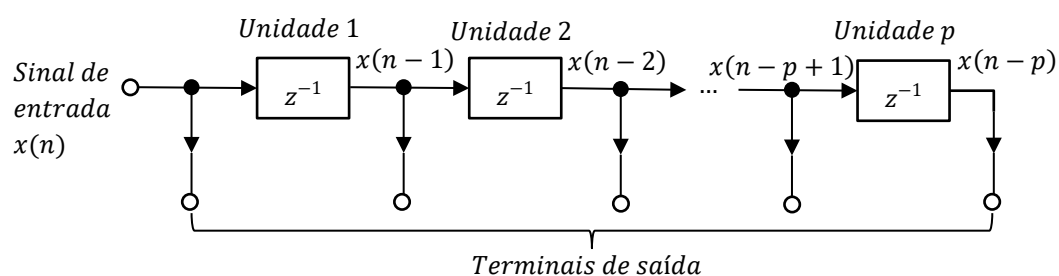
Em algumas aplicações, pode ser interessante que facultemos esse tipo de operação, dito processamento temporal, especialmente em processamento de fala, quando a predição do próximo fonema depende, em boa parte, do anterior, uma rede temporal pode ser mais útil e eficiente. No caso de RL, a temporalização da rede é de nosso especial interesse, uma vez que nos possibilitaria manter uma memória marcoviana dos estados do agente.

Duas são as formas de incorporarmos o tempo na operação de uma RNA. Podemos alterar a arquitetura de uma MLP, agregando uma memória de curto prazo externa ou criando uma rede recorrente. Vamos, a seguir, estudar ambas as possibilidades.

4.1 Processamento temporal em redes alimentadas adiante

Uma MLP clássica apresenta ao menos uma camada, exceto a de entrada. A camada de saída, nesse caso, é totalmente conectada com a de entrada, ou seja, cada saída de um neurônio da camada de entrada é conectada com uma entrada de cada neurônio da camada exposta. Podemos alterar um pouco essa topologia pelo acréscimo de uma “linha de atraso”, conforme se vê na Figura 8 a seguir.

Figura 8 – Linha de atraso



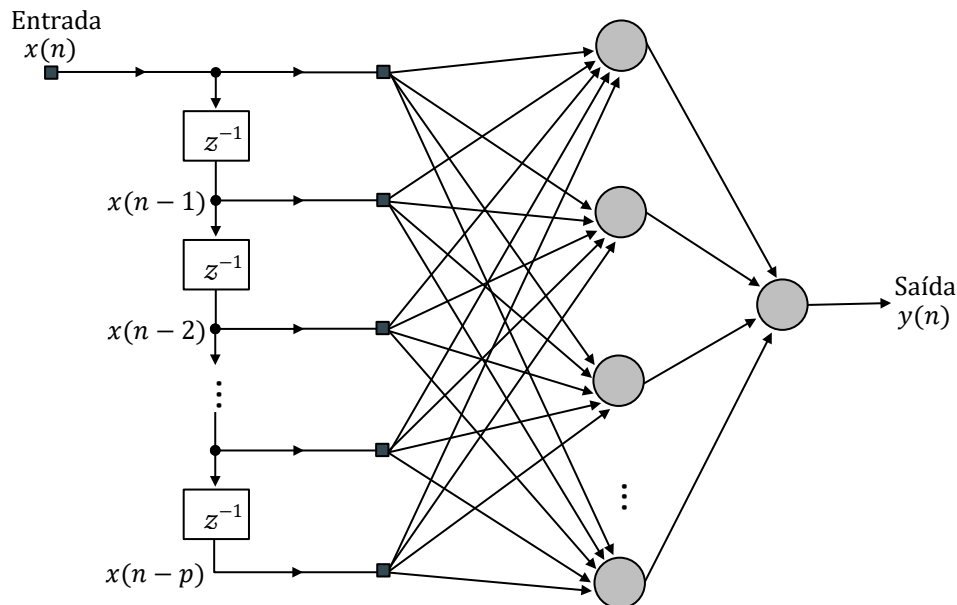
Fonte: Haykin, 2011, p. 689.

Essa linha baseia-se no atraso temporal inserido em uma função pela transformada z . Não mergulharemos na matemática envolvida, uma vez que, nas aplicações computacionais, basta nos retermos à informação anterior em uma memória do sistema. O sinal $x(n-1)$ é o vetor de teste que antecedeu a $x(n)$, da mesma forma que $x(n-2)$ é o vetor que antecedeu a $x(n-1)$.



Podemos agora agregar essa estrutura de atraso à entrada da rede neural, como se vê na Figura 9 a seguir.

Figura 9 – MLP com linha de atraso



Fonte: Haykin, 2011, p. 695.

O treinamento de uma rede, como a representada na Figura 9 anterior, pode ser feito por retropropagação de erro, como aprendemos em momento anterior. Naturalmente será necessário “carregar” a memória antes de cada propagação para frente. Após a propagação, segue-se o algoritmo padrão, comparando com o valor desejado e retropropagando o erro camada a camada.

Para modelos de Markov de ordens baixas e com vetores de entrada de baixa dimensionalidade, o modelo de MLP temporal pode ser útil. Conforme a ordem cresce, ou seja, quando exigimos memórias mais antigas, a complexidade do algoritmo de BP cresce, mas o problema se torna exponencial quando precisarmos prever processos multivariados temporais.

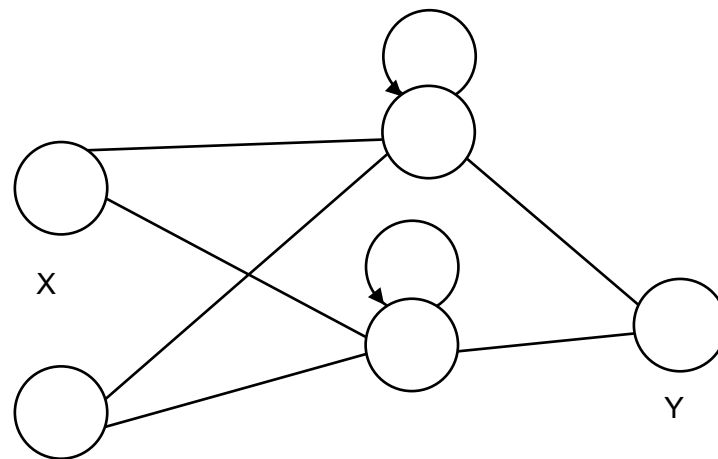
4.2 Processamento temporal neurodinâmico

Outra forma de agregar uma memória de curto prazo em uma rede neural é a inserção de recorrência. As redes que apresentam recorrência em um ou mais neurônios serão ditas RNR (redes neurais recorrentes) ou RNN (*recurrent neural network*, em inglês).

A recorrência em RNRs pode ser local ou global. As recorrências locais são obtidas pela reinserção do sinal de saída de um neurônio em uma de suas próprias entradas; já a recorrência global se dá pela reinserção do sinal de saída da rede em uma camada anterior. A recorrência global é de estabilidade complexa e o estudo dessa estabilidade é chamado de neurodinâmica, a qual não fará parte de nossos estudos.

Na Figura 10 a seguir, ilustramos a recorrência local de camada oculta.

Figura 10 – RNR local

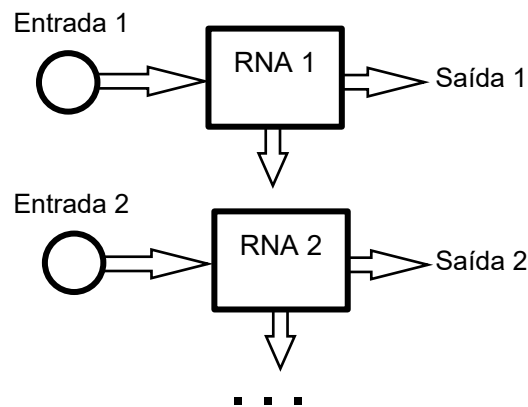


Fonte: Brustolin, 2022.

Observando a rede da figura e supondo uma evolução discreta do tempo em períodos descontínuos $0, \Delta, \Delta+1, \Delta+2, \dots$ podemos imaginar no instante $t=0$ que um vetor de entrada $X(t=0)$ será propagado até a saída $Y(t=0)$ como em uma MLP tradicional, e poderemos, se desejarmos, comparar o valor de Y com D , calculando $\mathcal{E}$.

Logo que atingimos Δ , entretanto, a camada oculta apresentará um comportamento distinto para $X(\Delta)$ que o presente em $t = 0$. Nesse instante, a saída dos neurônios ocultos de $t = 0$ será reinserida na entrada desses mesmos neurônios. Haverá uma memória do instante anterior, modificando o próximo instante. Dessa forma, podemos converter a rede temporal como uma sequência desdobrada de redes atemporais. A cada intervalo Δ , uma nova rede é criada para tratar o vetor de entrada, como ilustrado na Figura 11 a seguir.

Figura 11 – RNR desdobrada



Crédito: Brustolin 2022/UT

Vejamos que a rede que trata a “Entrada 1” não é a mesma que trata a “Entrada 2”, uma vez que parte do sinal de “1” é reinserido em “2”. Essa forma de entender as RNNs nos ajudará a estudar seu treinamento por BP, assunto que abordaremos a seguir.

4.3 Retropropagação temporal

A ideia de se desdobrar uma RNN de forma a podermos tratá-la como uma MLP tradicional nos traz várias vantagens. A capacidade de treinar a rede por BP é uma delas. Para tanto, precisamos entender que o erro se torna temporal, ou seja, é necessário considerar a variação temporal do erro. Outro fator importante a se considerar é que, além da matriz W que multiplica X , há uma segunda matriz temporal de W_t que representa os pesos sinápticos das realimentações temporais. Para entendermos W_t , basta imaginar que, quando um neurônio recebe sua própria saída, um peso sináptico estará associado a ela.

Na retropropagação em MLP, o cálculo da variação de w para os neurônios de saída diferiam do cálculo das camadas ocultas, posto que, nestas, não se conhece o sinal de erro.

Naquela oportunidade, afirmamos que a soma ponderal em relação a w dos gradientes de erro dos neurônios de saída poderiam ser utilizadas para calcular o Δw dos neurônios ocultos.

Da mesma forma, na retropropagação temporal (BPTT, *Back Propagation Through Time*, em inglês), considerando a RNN desdobrada, será necessário somar ponderalmente todos os gradientes temporais até o final da época.



4.4 Problema da dissipação da memória

Como acabamos de estudar, na BPTT somamos os erros temporais para o cálculo do gradiente de erro. No início da época de treinamento, o efeito do erro temporal será significativo, mas conforme nos aproximamos do final dos padrões, a multiplicação e diferenciação sucessiva, principalmente em funções de ativação sigmóides, termina por varrer da memória os primeiros padrões, tornando o processo bom apenas para baixas ordens de Markov.

O processo de BPTT apresenta uma memória próxima, mas não é capaz de guardar resquícios significativos de memórias distantes. Esse é um fenômeno bastante estudado em aprendizagem profunda e recebe o nome de *Vanishing Gradient Problem* (VGP).

Saiba mais

Não será nosso escopo aprofundar a solução do VGP, apenas indicaremos o artigo de Hochreiter e Schmidhuber (1997), que introduziu as redes LSTM (*Long Short-Term Memory*), as quais possibilitaram uma boa retenção de eventos pregressos.

TEMA 5 – REDES CONVOLUCIONAIS

Uma das questões problemáticas em treinamento de redes neurais se refere à incorporação de invariâncias à memória da rede. As redes alimentadas adiante têm eficiência sofrível no reconhecimento de invariâncias. O exemplo mais contundente se dá em reconhecimento de imagens por computador. Uma imagem não pode ter sua classificação de saída alterada em função da rotação ou de varrições de luminosidade ou mesmo da localização da imagem no *frame* consumido pela máquina.

Redes neurais convolucionais (CNN) têm uma arquitetura que possibilita a incorporação dessas invariâncias.

5.1 Histórico da arquitetura

O trabalho de Hubel e Wiesel (1968, p. 215 e seguintes) demonstrou que o córtex visual animal tem diversas organizações celulares sensíveis distintas. Esses conjuntos celulares são ativados conforme a complexidade da região da imagem que os sensibiliza.



Conjuntos de células simples são sensibilizadas quando observam padrões simples, a exemplo de linhas. Em contrapartida, conjuntos complexos são sensibilizados por combinações de padrões simples.

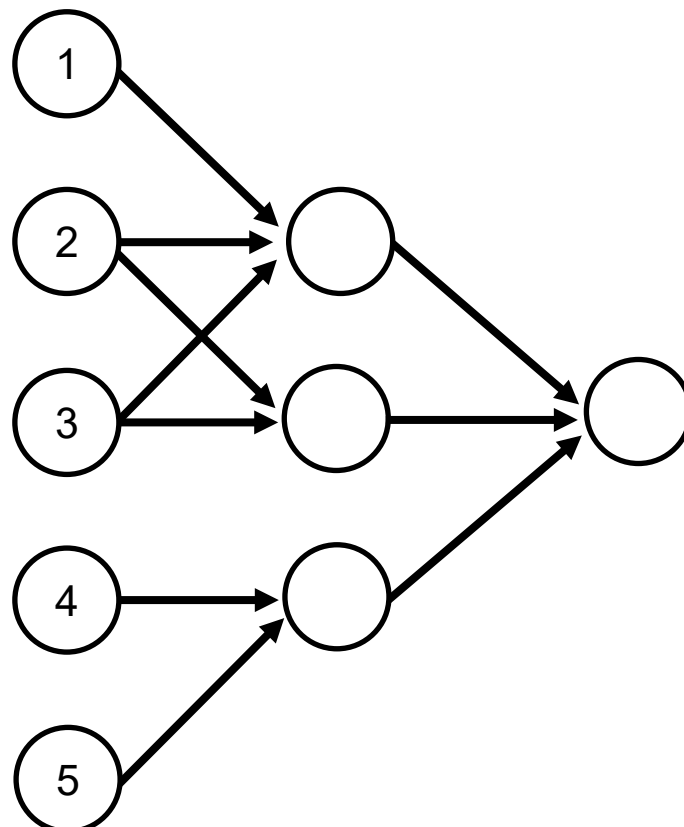
Com base no estudo de Hubel, elabora-se a ideia hierárquica de representação de imagens. Passamos a entender imagens complexas como combinações de elementos mais simples como arestas e não apenas pela matriz composta pelos elementares da imagem, os *pixels*.

Do ponto de vista de implementação, o empilhamento de camadas mais se assemelha à conclusão acima do que imensas camadas de entrada. Da mesma forma, a plena conexão entre neurônios de camadas sucessivas, típica da MLPs, não faz sentido nessa arquitetura, uma vez que buscamos especializar conjuntos de neurônios.

5.2 Mapa de características

Na Figura 12 a seguir, ilustramos uma CNN elementar. Vamos observar que os neurônios de entrada são seletivamente conectados aos neurônios ocultos.

Figura 12 – CNN elementar



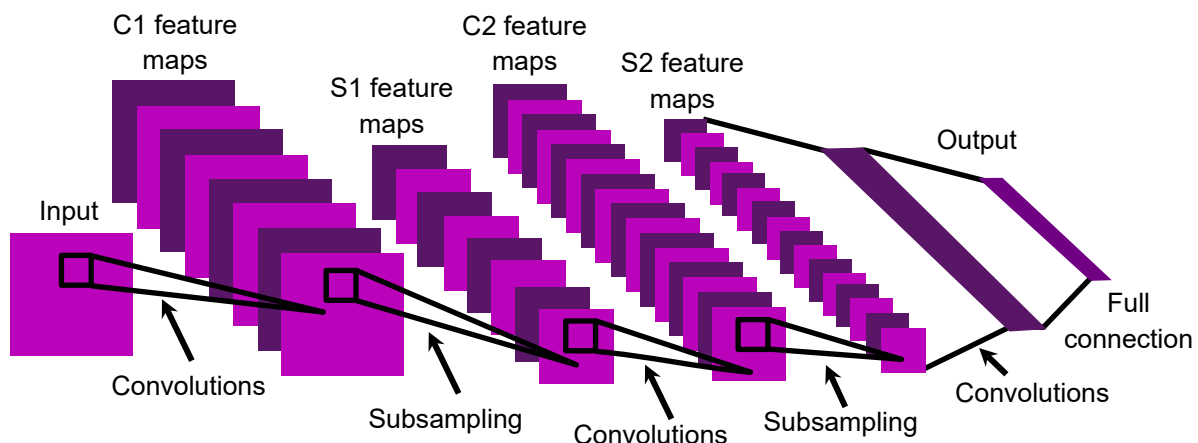
Fonte: Brustolin, 2022.



As CNN geralmente dedicam conjuntos de neurônios por regiões da imagem. A camada de entrada captura alguns *pixels*, associando-os à camada oculta, segundo campos receptivos. Na Figura 12 anterior, podemos ver que tanto o neurônio de entrada 2 quanto 3 estão conectados aos mesmos neurônios da camada oculta, configurando um campo elementar.

Cada campo se torna um filtro não linear cujos pesos são ajustados para que o neurônio seja ativado apenas se estiver presente o estímulo do padrão. A CNN amostrará então toda a imagem, aplicando os filtros neurais, criando um mapa de características (*feature maps*) da imagem “convolucionada”. Novas amostragens podem ser feitas sobre os *feature maps*, concolucionando a imagem uma segunda vez. Na Figura 13 a seguir, ilustramos o procedimento para dois estágios. Nesse exemplo, são aplicados oito filtros convolucionais em C1, logo, diremos que a CNN tem profundidade 8 no primeiro estágio de convolução.

Figura 13 – CNN



Fonte: Lecun; Kavukcuoglu; Farabet, 2010.

Observamos que, após o primeiro estágio convolucional, os mapas são reduzidos. Esse estágio é denominado *pooling*, uma vez que amostra os mapas em busca de características essenciais, desprezando outras, menos importantes, como a localização de determinado mapa no *frame*. Após as convoluções e *poolings*, os mapas são classificados por uma RNA alimentada adiante.



5.3 Deep Q-Learning Network

As redes neurais não são apenas aplicáveis em reconhecimento de imagem, mas também podem ser engenhadas para auxiliar em processos de aprendizagem por reforço. De maneira especial, são um bom recurso para a estimativa da política ótima via maximização do valor da ação, Q .

A determinação da política ótima em *reinforcement learning*, como já sabemos, embora tenha encontrado um formalismo matemático a contento pelas equações de Bellman, não tem implementação simples. Métodos recorrentes são normalmente escolhidos para equacionar o problema.

A escolha de métodos iterativos nos remete, mesmo que inconscientemente, às redes neurais e ao seu treinamento iterativo por retropropagação de erro. Vamos tecer algumas considerações matemáticas para comprovar essa intuição.

Tomando a equação recorrente para cálculo de $Q_{\max}$ em sistemas estocásticos temporais:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Podemos encarar $\alpha(r + \gamma \max_{a'} Q(s', a'))$ como o valor alvo de nossa iteração ao passo que $Q(s, a)$ é o valor da predição atual. Paramos as iterações para o cálculo de $Q^*(s, a)$ quando o valor de $Q(s, a)$ apresenta variação suficientemente baixa. Esse processo é o equivalente à minimização da segunda parte da equação.

Uma forma de minimizar a distância entre $\alpha(r + \gamma \max_{a'} Q(s', a'))$ e $Q(s, a)$ é pela minimização da distância quadrática, por gradiente descendente.

Se fizermos essa consideração, o uso de redes neurais se torna uma opção natural para o cálculo da política ótima. Mas qual das arquiteturas melhor se encaixaria na estimativa de parâmetros?

Se tomarmos o ambiente como uma imagem a ser entendida pelo agente, podemos convolucionar o ambiente para extrair suas características e então maximizar a função valor do estado.

Naturalmente esse é um tema excessivamente complexo para que o tratemos nesse ponto de nosso estudo, basta-nos, por hora, entender a aplicabilidade das RNAs também em RL.



FINALIZANDO

Nesta etapa, mergulhamos nas redes neurais artificiais para entendermos como seria possível solucionar um problema de aprendizado em ambiente estocástico tanto em meios atemporais quanto sequenciais, pelo uso dessas redes.

Para chegar a esse ponto, percorremos um virtuoso caminho, entendendo o modelamento do neurônio e as arquiteturas básicas de redes. Compreendemos também de que forma uma RNA adquire memória, como pode ser treinado e as limitações de generalização. Finalmente, fizemos um breve passeio em arquiteturas temporais e redes convolucionais. Os tópicos foram estudados com a profundidade mínima necessária para a compreensão das aplicações possíveis dessa ferramenta de IA. Para aqueles que eventualmente desejam operacionalizar essas redes, recomendamos a leitura das obras referenciadas ao longo do conteúdo.



REFERÊNCIAS

CASTOLDI, M. F. **Algoritmo híbrido para projeto de controladores de amortecimento de sistemas elétricos de potência utilizando algoritmos genéticos e gradiente descendente**. Tese (Doutorado) – Universidade de São Paulo, São Paulo, 2011.

HAYKIN, S. **Redes neurais**. Porto Alegre: Grupo A, 2011. Disponível em: <<https://integrada.minhabiblioteca.com.br/#/books/9788577800865/>>. Acesso em: 15 maio 2022.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural computation**, v. 9, n. 8, p. 1.735-1.780, 1997. Disponível em: <<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.676.4320&rep=rep1&type=pdf>>. Acesso em: 15 maio 2022.

HUBEL, D. H.; WIESEL, T. N. Receptive fields and functional architecture of monkey striate cortex. **The Journal of Physiology**, v. 195, n. 1, p. 215-243, mar. 1968. Disponível em: <<https://physoc.onlinelibrary.wiley.com/doi/abs/10.1113/jphysiol.1968.sp008455>>. Acesso em: 15 maio 2022.

LECUN, Y.; KAVUKCUOGLU, K.; FARABET, C. Convolutional networks and applications in vision. In: INTERNATIONAL SYMPOSIUM ON CIRCUITS AND SYSTEMS, Paris. **Anais...** Paris, 2010. p. 253-256.

MCCALLUM, W. G. **Cálculo de várias variáveis**. São Paulo: Blucher, 1997. Disponível em: <<https://integrada.minhabiblioteca.com.br/#/books/9788521217879/>>. Acesso em: 15 maio 2022.

NORVIG, P. **Inteligência artificial**. 3. ed. Porto Alegre: Grupo GEN, 2013.